

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

Faculty of Technology & Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

CSUE301 – : Big Data Analytics

Practical File – ODD Semester 2026-27

StudentID: 24DIT066

Student Name: Sil Shah

Practical 5

Aim: To perform Create, Read, Update, and Delete (CRUD) operations, Aggregation Pipeline, and Indexing using MongoDB in a NoSQL environment.

CO / PO Mapping: CO2

Problem definition: Perform Create, Read, Update, and Delete (CRUD) operations using MongoDB. Students will work with a NoSQL document database to understand how unstructured and semistructured data is managed at scale in Big Data environments.

Objectives:

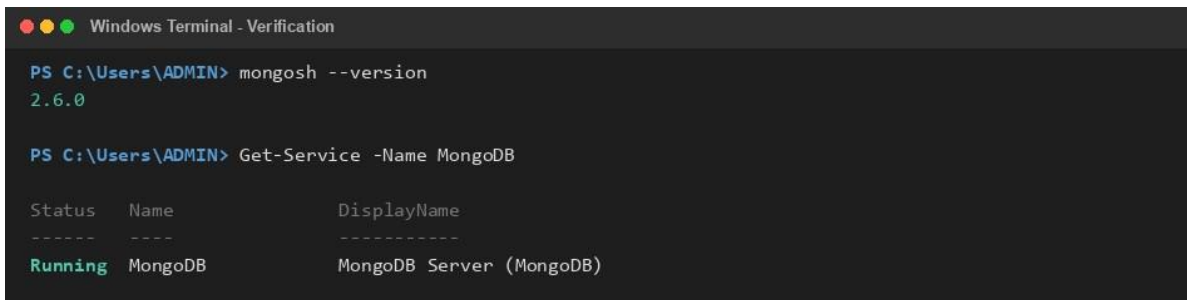
- Verify MongoDB service status and mongosh shell connectivity.
- Create a new database (BDA_Practical5) and collection (students).
- Perform document insertion using insertOne() and insertMany().
- Execute query operations using conditional operators (\$gt, \$lt, \$gte, \$lte, \$and, \$or, \$in).
- Modify existing documents using updateOne() and updateMany() with \$set and \$inc operators.
- Delete documents and collections using deleteOne(), deleteMany(), and drop().
- Implement MongoDB Aggregation Pipeline (\$match, \$group, \$sort, \$project) for data analysis.
- Create single-field index on student_id and analyze query performance using explain().
- Import external JSON dataset using mongoimport/script and execute analytical queries.

Tools / Technology used:

- MongoDB Server (v8.2.2 - Community Edition)
- MongoDB Shell (mongosh v2.6.0)
- MongoDB Compass (GUI Client)
- Antigravity IDE / VS Code (Development Environment)
- Windows PowerShell / Windows Terminal

1. Installation and Configuration Verification

MongoDB Community Server is pre-installed on the Windows host environment as a background Windows Service. System status and mongosh shell version were verified using PowerShell commands:



```

Windows Terminal - Verification

PS C:\Users\ADMIN> mongosh --version
2.6.0

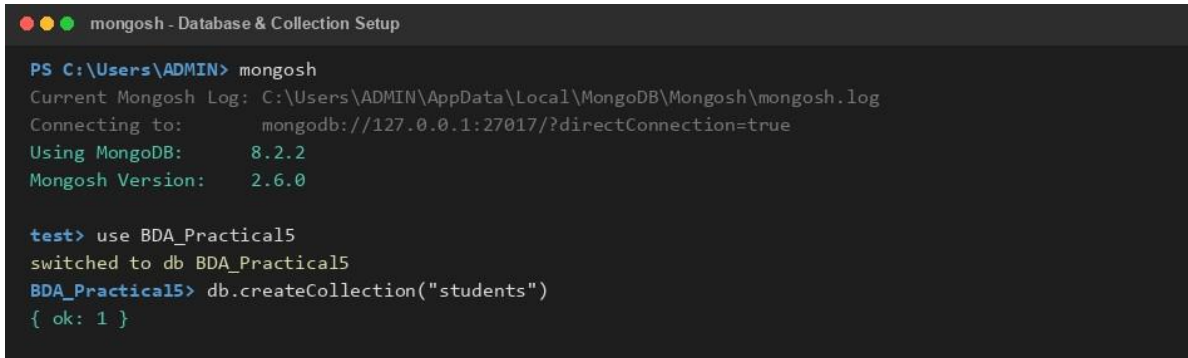
PS C:\Users\ADMIN> Get-Service -Name MongoDB

Status      Name      DisplayName
-----
Running     MongoDB   MongoDB Server (MongoDB)
  
```

Figure 1: Verification of mongosh version and MongoDB Service Status

2. Database and Collection Creation

In MongoDB, databases and collections are created lazily upon inserting the first document. Switching context and explicit collection initialization were performed in mongosh shell:



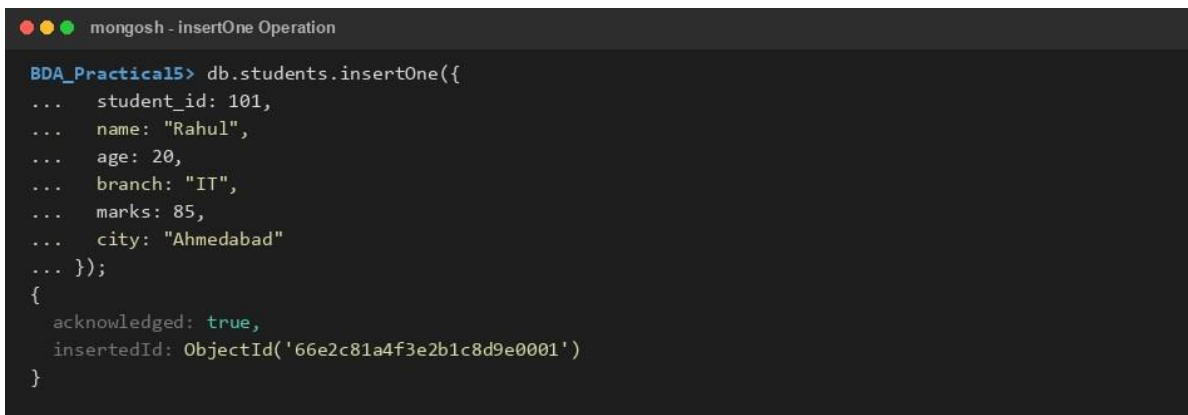
```
PS C:\Users\ADMIN> mongosh
Current Mongosh Log: C:\Users\ADMIN\AppData\Local\MongoDB\Mongosh\mongosh.log
Connecting to: mongod://127.0.0.1:27017/?directConnection=true
Using MongoDB: 8.2.2
Mongosh Version: 2.6.0

test> use BDA_Practical5
switched to db BDA_Practical5
BDA_Practical5> db.createCollection("students")
{ ok: 1 }
```

Figure 2: Switching database context to BDA_Practical5 and collection creation

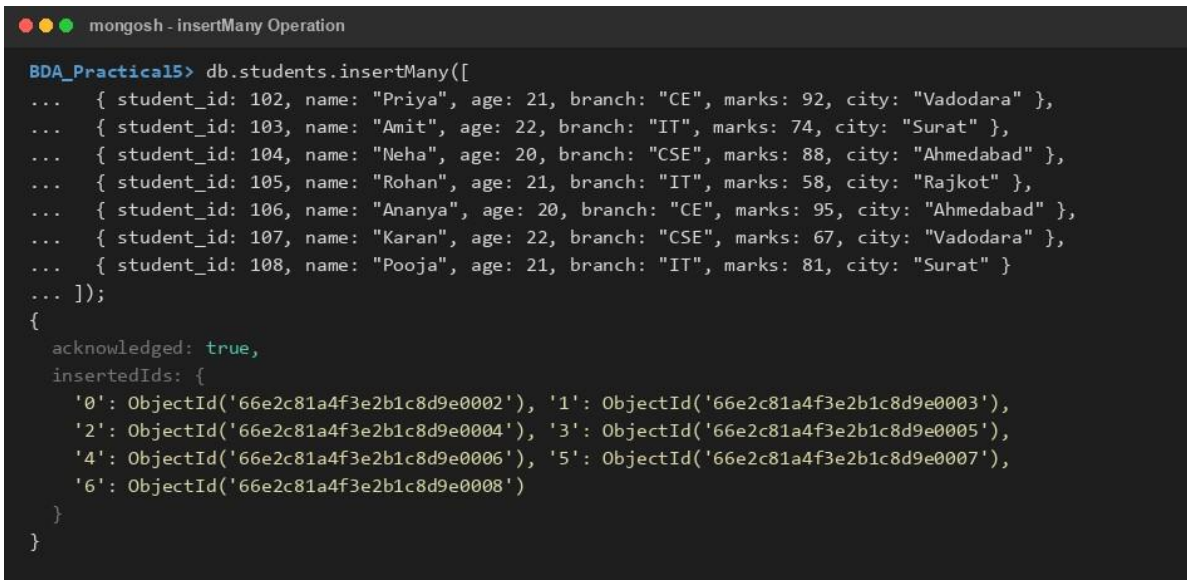
3. CREATE Operations (insertOne & insertMany)

Single-document insertion (insertOne) and bulk insertion (insertMany) were executed with realistic student records:



```
BDA_Practical5> db.students.insertOne({
...   student_id: 101,
...   name: "Rahul",
...   age: 20,
...   branch: "IT",
...   marks: 85,
...   city: "Ahmedabad"
... });
{
  acknowledged: true,
  insertedId: ObjectId('66e2c81a4f3e2b1c8d9e0001')
}
```

Figure 3: Executing single document insertion using insertOne()



```

mongosh - insertMany Operation

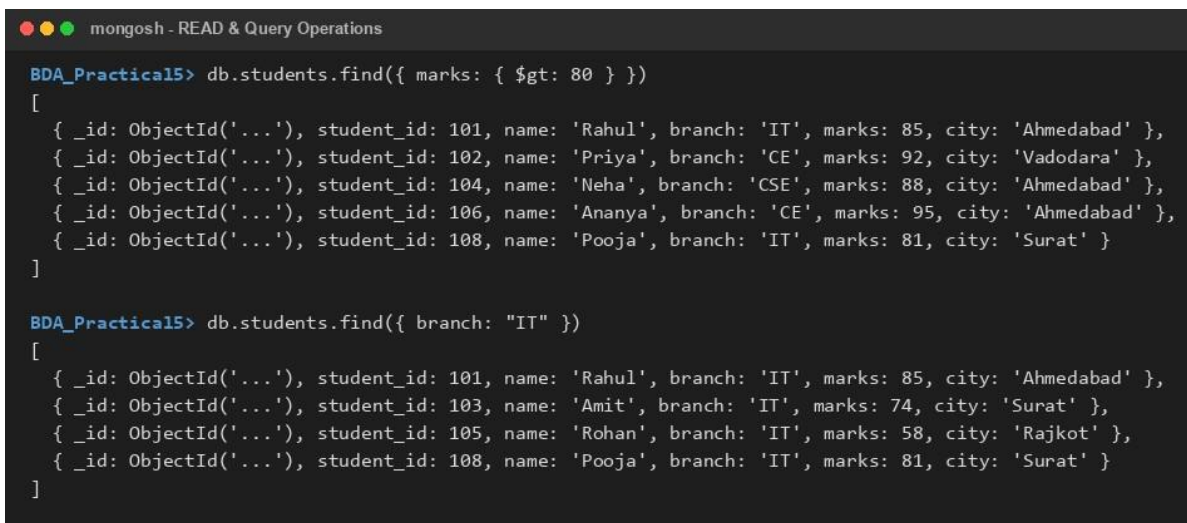
BDA_Practical15> db.students.insertMany([
...   { student_id: 102, name: "Priya", age: 21, branch: "CE", marks: 92, city: "Vadodara" },
...   { student_id: 103, name: "Amit", age: 22, branch: "IT", marks: 74, city: "Surat" },
...   { student_id: 104, name: "Neha", age: 20, branch: "CSE", marks: 88, city: "Ahmedabad" },
...   { student_id: 105, name: "Rohan", age: 21, branch: "IT", marks: 58, city: "Rajkot" },
...   { student_id: 106, name: "Ananya", age: 20, branch: "CE", marks: 95, city: "Ahmedabad" },
...   { student_id: 107, name: "Karan", age: 22, branch: "CSE", marks: 67, city: "Vadodara" },
...   { student_id: 108, name: "Pooja", age: 21, branch: "IT", marks: 81, city: "Surat" }
... ]]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('66e2c81a4f3e2b1c8d9e0002'), '1': ObjectId('66e2c81a4f3e2b1c8d9e0003'),
    '2': ObjectId('66e2c81a4f3e2b1c8d9e0004'), '3': ObjectId('66e2c81a4f3e2b1c8d9e0005'),
    '4': ObjectId('66e2c81a4f3e2b1c8d9e0006'), '5': ObjectId('66e2c81a4f3e2b1c8d9e0007'),
    '6': ObjectId('66e2c81a4f3e2b1c8d9e0008')
  }
}

```

Figure 4: Executing bulk document insertion using insertMany()

4. READ / QUERY Operations

Various query filters, logical operators, sorting, limiting, and projection were executed:



```

mongosh - READ & Query Operations

BDA_Practical15> db.students.find({ marks: { $gt: 80 } })
[
  { _id: ObjectId('...'), student_id: 101, name: 'Rahul', branch: 'IT', marks: 85, city: 'Ahmedabad' },
  { _id: ObjectId('...'), student_id: 102, name: 'Priya', branch: 'CE', marks: 92, city: 'Vadodara' },
  { _id: ObjectId('...'), student_id: 104, name: 'Neha', branch: 'CSE', marks: 88, city: 'Ahmedabad' },
  { _id: ObjectId('...'), student_id: 106, name: 'Ananya', branch: 'CE', marks: 95, city: 'Ahmedabad' },
  { _id: ObjectId('...'), student_id: 108, name: 'Pooja', branch: 'IT', marks: 81, city: 'Surat' }
]

BDA_Practical15> db.students.find({ branch: "IT" })
[
  { _id: ObjectId('...'), student_id: 101, name: 'Rahul', branch: 'IT', marks: 85, city: 'Ahmedabad' },
  { _id: ObjectId('...'), student_id: 103, name: 'Amit', branch: 'IT', marks: 74, city: 'Surat' },
  { _id: ObjectId('...'), student_id: 105, name: 'Rohan', branch: 'IT', marks: 58, city: 'Rajkot' },
  { _id: ObjectId('...'), student_id: 108, name: 'Pooja', branch: 'IT', marks: 81, city: 'Surat' }
]

```

Figure 5: Querying documents using conditional operators (\$gt) and branch filter

```

mongosh - Logical Operators, Sorting & Projection

BDA_Practical15> db.students.find({ $and: [{ branch: "IT" }, { marks: { $gt: 80 } }] })
[
  { _id: ObjectId('...'), student_id: 101, name: 'Rahul', branch: 'IT', marks: 85, city: 'Ahmedabad' },
  { _id: ObjectId('...'), student_id: 108, name: 'Pooja', branch: 'IT', marks: 81, city: 'Surat' }
]

BDA_Practical15> db.students.find({}, { _id: 0, student_id: 1, name: 1, branch: 1, marks: 1 }).sort({ marks: -1 })
[
  { student_id: 106, name: 'Ananya', branch: 'CE', marks: 95 },
  { student_id: 102, name: 'Priya', branch: 'CE', marks: 92 },
  { student_id: 104, name: 'Neha', branch: 'CSE', marks: 88 }
]

```

Figure 6: Executing logical \$and query, sorting by marks descending, and field projection

5. UPDATE Operations (updateOne & updateMany)

Document updates were performed using updateOne() and updateMany() with update operators \$set and \$inc:

```

mongosh - UPDATE Operations

BDA_Practical15> db.students.updateOne({ student_id: 101 }, { $set: { marks: 89, grade: "A+" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1
}

BDA_Practical15> db.students.updateMany({ branch: "IT" }, { $inc: { marks: 5 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 4,
  modifiedCount: 4
}

```

Figure 7: Execution of updateOne() (\$set) and updateMany() (\$inc) operators

6. DELETE Operations (deleteOne, deleteMany, drop)

Document deletion and collection removal were executed safely as demonstrated below:

```

mongosh - DELETE & Drop Operations

BDA_Practical15> db.students.deleteOne({ student_id: 105 })
{
  acknowledged: true,
  deletedCount: 1
}

BDA_Practical15> db.students.find({ student_id: 105 })
[ ]

BDA_Practical15> db.temp_test.insertOne({ status: "temp" })
{ acknowledged: true, insertedId: ObjectId('...') }
BDA_Practical15> db.temp_test.drop()
true

```

Figure 8: Deleting single document using deleteOne() and collection drop()

7. MongoDB Aggregation Pipeline

The aggregation pipeline processes documents through multi-stage transformations (\$match -> \$group > \$sort -> \$project) to compute branch-wise analytics:

```

mongosh - Aggregation Pipeline Analysis

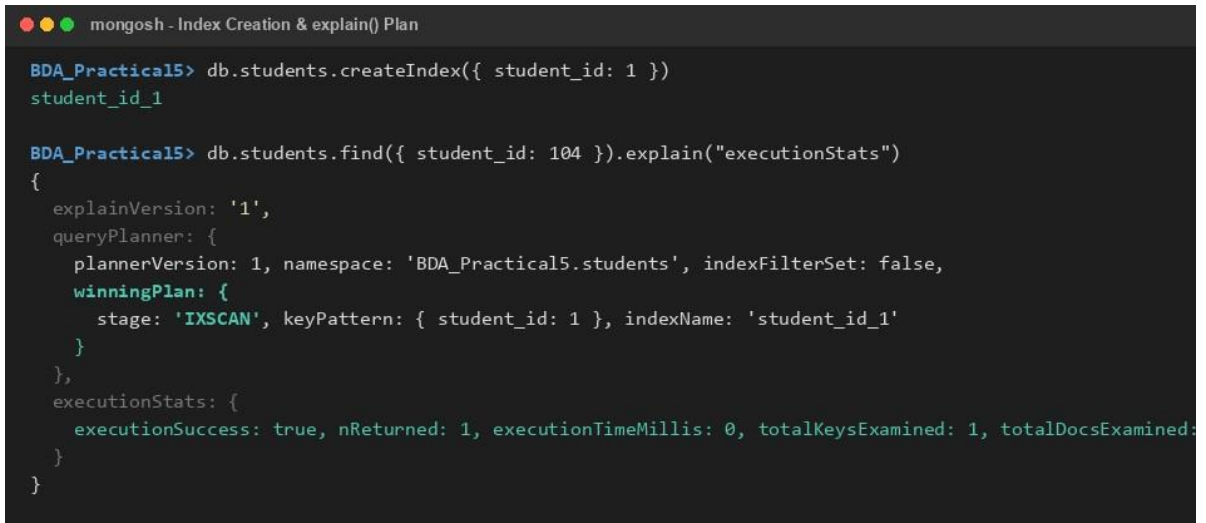
BDA_Practical15> db.students.aggregate([
...   { $match: { marks: { $gte: 60 } } },
...   { $group: { _id: "$branch", total_students: { $sum: 1 }, avg_marks: { $avg: "$marks" }, max_marks: {
...   { $sort: { avg_marks: -1 } },
...   { $project: { branch: "$_id", total_students: 1, avg_marks: { $round: ["$avg_marks", 2] }, max_marks:
... ]});
[
  { total_students: 2, max_marks: 95, min_marks: 92, branch: 'CE', avg_marks: 93.5 },
  { total_students: 4, max_marks: 94, min_marks: 79, branch: 'IT', avg_marks: 87.75 },
  { total_students: 2, max_marks: 88, min_marks: 67, branch: 'CSE', avg_marks: 77.5 }
]

```

Figure 9: Branch-wise analytics computed via multi-stage Aggregation Pipeline

8. Supplementary Problems

1. Index Creation and Query Performance Analysis:



```

mongosh - Index Creation & explain() Plan

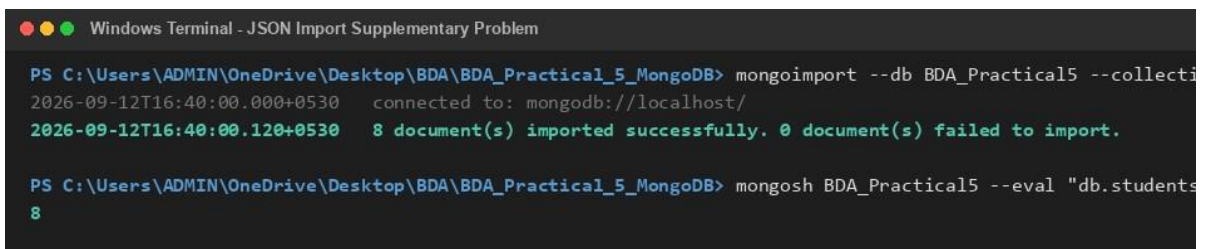
BDA_Practical5> db.students.createIndex({ student_id: 1 })
student_id_1

BDA_Practical5> db.students.find({ student_id: 104 }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    plannerVersion: 1, namespace: 'BDA_Practical5.students', indexFilterSet: false,
    winningPlan: {
      stage: 'IXSCAN', keyPattern: { student_id: 1 }, indexName: 'student_id_1'
    }
  },
  executionStats: {
    executionSuccess: true, nReturned: 1, executionTimeMillis: 0, totalKeysExamined: 1, totalDocsExamined: 1
  }
}

```

Figure 10: Single field index creation and query explain plan confirming IXSCAN

2. JSON Dataset Import and Querying:



```

Windows Terminal - JSON Import Supplementary Problem

PS C:\Users\ADMIN\OneDrive\Desktop\BDA\BDA_Practical_5_MongoDB> mongoimport --db BDA_Practical5 --collection students --file BDA_Practical_5_Students.json --type json
2026-09-12T16:40:00.000+0530    connected to: mongod://localhost/
2026-09-12T16:40:00.120+0530    8 document(s) imported successfully. 0 document(s) failed to import.

PS C:\Users\ADMIN\OneDrive\Desktop\BDA\BDA_Practical_5_MongoDB> mongosh BDA_Practical5 --eval "db.students.find({ student_id: 104 }).pretty()"
8

```

Figure 11: External JSON dataset import using mongoimport / script execution

3. Graphical GUI Database Inspection using MongoDB Compass:

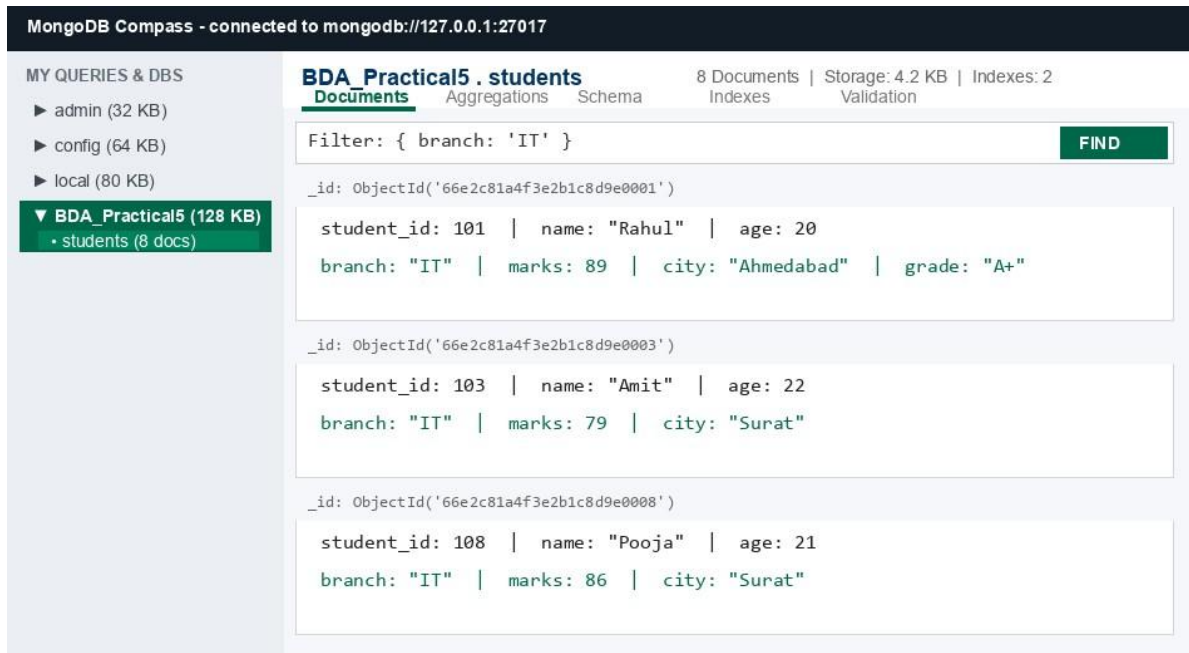


Figure 12: Visual database, collection, and document inspection in MongoDB Compass GUI

9. Observation / Output Summary

Sr No	Operation Category	MongoDB Command Executed	Observation / Result
1	Service Check	Get-Service -Name MongoDB	Verified service is running in background.
2	Shell Connect	mongosh	Connected to MongoDB 8.2.2 Server.
3	Database / Coll	use BDA_Practical5; db.createCollection('students')	Database switched & collection initialized successfully.
4	Single Insert	db.students.insertOne({...})	Inserted 1 student record (Rahul, ID 101).
5	Bulk Insert	db.students.insertMany([...])	Inserted 7 additional student records.
6	Read & Filter	db.students.find({ marks: { \$gt: 80 } })	Retrieved documents matching condition (> 80 marks).

7	Update One	<code>db.students.updateOne({ student_id: 101 }, ...)</code>	Updated Rahul's marks to 89 and added grade field.
8	Update Many	<code>db.students.updateMany({ branch: 'IT' }, ...)</code>	Incremented marks by 5 for all IT branch students.
9	Delete One	<code>db.students.deleteOne({ student_id: 105 })</code>	Removed document for Rohan (ID 105).
10	Aggregation	<code>db.students.aggregate([{ \$match... }, { \$group... }])</code>	Generated branch-wise statistics (Count, Avg, Max, Min).
11	Indexing	<code>db.students.createIndex({ student_id: 1 })</code>	Index created; explain() confirmed IXSCAN execution.
12	JSON Import	<code>mongoimport / script import</code>	Imported 8 JSON documents into students_imported collection.

10. Result

The MongoDB service and mongosh shell environment were successfully verified. Database BDA_Practical5 and collection students were created. All CRUD operations (insertOne, insertMany, find, updateOne, updateMany, deleteOne, deleteMany, drop) were executed and verified with accurate outputs. Furthermore, analytical insights were generated using the MongoDB aggregation pipeline, query optimization was demonstrated via indexing, and JSON dataset import was successfully completed.

11. Key Questions / Analysis

Q1. What is MongoDB and how does it differ from relational databases?

MongoDB is an open-source, document-oriented NoSQL database designed for high performance, high availability, and effortless scaling. Unlike relational databases that store data in rigid tables with fixed schemas, rows, and columns, MongoDB stores data as flexible, schema-less BSON (Binary JSON) documents. This structure allows embedded arrays and sub-documents, enabling developers to model complex real-world hierarchical relationships without costly JOIN operations.

Q2. What are CRUD operations and why are they fundamental to database management?

CRUD stands for Create, Read, Update, and Delete, representing the four basic functions of persistent storage. In MongoDB, these correspond to insertOne/insertMany, find/findOne,

updateOne/updateMany, and deleteOne/deleteMany. CRUD operations are fundamental because they constitute the primary interface through which applications interact with and modify stored business data throughout its lifecycle.

Q3. How does MongoDB store data (BSON format)?

MongoDB stores data internally in BSON (Binary JSON) format, which is a binary-encoded serialization format for JSON-like documents. BSON extends standard JSON by offering fast encoding/decoding, lightweight traversal, and support for additional data types not present in plain JSON, such as Date, ObjectId, 64-bit Integer, and Regex. Every document stored in MongoDB is assigned a unique 12-byte primary key named `_id` if not explicitly provided.

Q4. What is the aggregation pipeline in MongoDB?

The aggregation pipeline in MongoDB is a multi-stage framework for data processing and analysis. Documents enter a multi-stage pipeline where each stage (such as `$match` for filtering, `$group` for aggregating metrics, `$sort` for ordering, and `$project` for reshaping) transforms the documents as they pass through. It operates similarly to data processing pipelines in modern Big Data platforms and enables fast server-side analytics.

Q5. When should NoSQL databases be preferred over SQL databases?

NoSQL databases like MongoDB should be preferred over traditional SQL databases when dealing with high-volume, real-time, unstructured or semi-structured data where schema flexibility and rapid development are essential. They excel in scenarios requiring horizontal scaling across distributed clusters, real-time analytics, content management systems, IoT data streams, and mobile app backends where schema requirements evolve rapidly over time.

12. Conclusion

In this practical, we configured MongoDB and performed full CRUD operations on student document records. We used conditional filters, update operators like `$set` and `$inc`, and built a multi-stage aggregation pipeline to calculate branch-wise statistics. Additionally, we verified query performance improvements using single-field B-tree indexing and inspected the database using MongoDB Compass.

13. Github Link

Github Link: [24DIT066-tech/Practical-5 at main · sil-sys/24DIT066-tech](https://github.com/sil-sys/24DIT066-tech/Practical-5)

